STAC AI BOOTCAMP NYC



Retrieval and RAG

Matthew Dixon, PhD, FRM
BeliefLens

21 October 2026 | Principles, algorithms and evaluation

matthew.dixon@belieflens.org

Why retrieval knowledge still matters

- An LLM can interpret text that you supply.
- Your system still determines which information it sees.
- Understanding retrieval helps locate missing, irrelevant or misleading context.

Decision: Know whether to change the evidence pipeline or the model.

Python notebook: `02_grounded_rag.ipynb`. Start with C01–C06; no model key required.

A roadmap driven by design questions

Question	Concepts we need
What information can be searched?	Collection, passage units, metadata and indexes.
How do we find candidates?	Lexical and learned representations.
Which candidates reach the model?	Hybrid retrieval, reranking and context budgets.
How do we know it works?	Grounding, component metrics and controlled tests.

Decision: Follow the information through the system before selecting tools.

Five capabilities to develop

- Specify the retrieval problem and eligible collection.
- Explain how lexical and dense ranking work.
- Choose a retrieval and reranking architecture.
- Construct a bounded, inspectable model context.
- Evaluate retrieval and generation separately.

Decision: Turn each capability into an observable design or diagnostic task.

Define the problem before the scoring function — 1 of 3

1. Start with the information need	Specify what must be established: for example, the current cancellation notice period for a particular agreement.

Why this matters: The task defines relevance; a retrieval model estimates which items may satisfy it.

Define the problem before the scoring function — 2 of 3

1. Start with the information need	Specify what must be established: for example, the current cancellation notice period for a particular agreement.
2. Construct a search query	The application or model expresses that need as search text. Preserve the entity, date and other constraints.

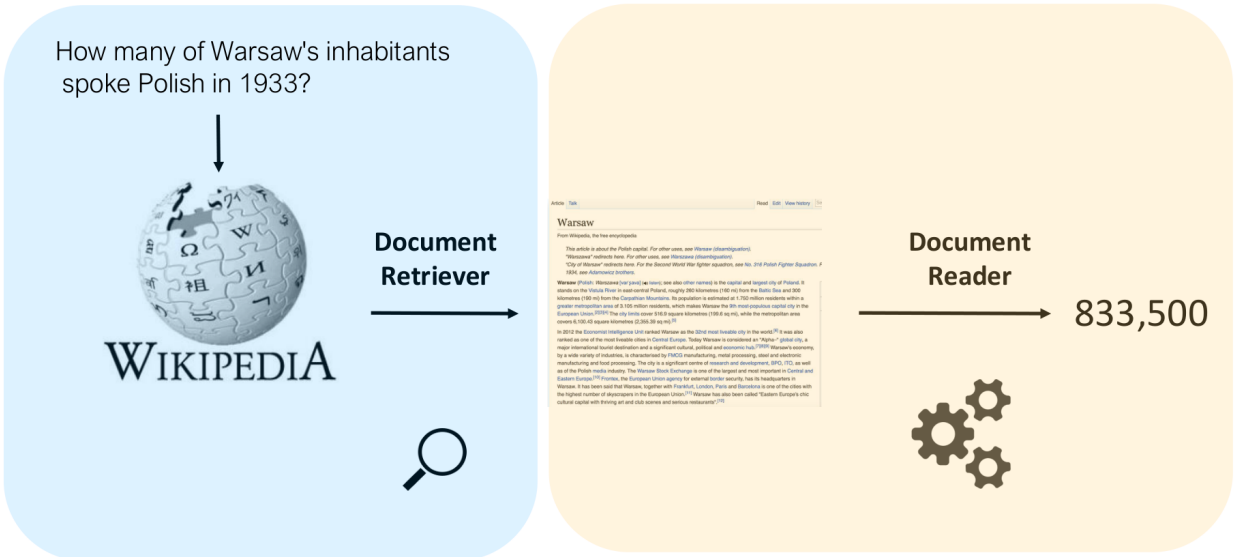
Why this matters: The task defines relevance; a retrieval model estimates which items may satisfy it.

Define the problem before the scoring function — 3 of 3

1. Start with the information need	Specify what must be established: for example, the current cancellation notice period for a particular agreement.
2. Construct a search query	The application or model expresses that need as search text. Preserve the entity, date and other constraints.
3. Judge relevance against the need	A passage is useful if it helps answer that scoped question. A high score alone does not make an old clause applicable.

Why this matters: The task defines relevance; a retrieval model estimates which items may satisfy it.

The retriever and reader solve different problems



Retriever

Select passages from a collection.

Reader / generator

Answer using the selected passages and question.

Measure both stages.

Decision: A retrieval miss and a reading error require different repairs.

Build once; retrieve many times

Collection preparation	Per-query work
Acquire and parse source versions.	Interpret the question and apply access scope.
Choose passage boundaries and preserve metadata.	Retrieve candidates and rerank.
Build lexical and/or vector indexes.	Select context and generate an answer.
Track additions, changes and deletions.	Record the source versions actually used.

Decision: Separate ingestion failures from query-time failures.

Choose what one searchable item means

Retrieval unit	Trade-off
Whole document	More context; a relevant section can be diluted.
Passage or section	More focused; may lose dependencies outside the boundary.
Sentence or field	Precise; can omit qualifications and referents.

Decision: Choose a unit that preserves the information needed for the target questions.

Chunking trades focus against context

- Small chunks improve focus but may split a condition from its conclusion.
- Large chunks retain context but consume budget and include unrelated text.
- Overlap reduces boundary loss while increasing duplicate content.
- Evaluate boundaries using answer-bearing spans and their qualifications.

Decision: Test chunking against information loss, not a universal token count.

Filters define eligibility; scores order candidates — 1 of 3

1. Check whether a source may be used	Access, scope and version rules determine eligibility. An old or restricted source cannot become eligible through a high score.

Why this matters: A relevant-looking passage still needs the right source, time and access status.

Filters define eligibility; scores order candidates — 2 of 3

1. Check whether a source may be used	Access, scope and version rules determine eligibility. An old or restricted source cannot become eligible through a high score.
2. Rank the eligible candidates	Keyword, vector or reranker scores prioritize potentially useful items. Ranking is separate from the hard source rules.

Why this matters: A relevant-looking passage still needs the right source, time and access status.

Filters define eligibility; scores order candidates — 3 of 3

1. Check whether a source may be used	Access, scope and version rules determine eligibility. An old or restricted source cannot become eligible through a high score.
2. Rank the eligible candidates	Keyword, vector or reranker scores prioritize potentially useful items. Ranking is separate from the hard source rules.
3. Keep uncertainty visible	If a version or permission is unknown, record the gap. Do not silently convert missing metadata into approval.

Why this matters: A relevant-looking passage still needs the right source, time and access status.

Notebook exercise: C02–C06. Inspect a passage, change the query, apply issuer/time filters.

Lexical retrieval depends on text representation

- Tokenization decides what counts as a term.
- Normalization can join variants but can also erase meaningful distinctions.
- Phrase and positional indexes preserve relationships beyond a bag of words.
- Negation, identifiers and punctuation may carry task-critical meaning.

Decision: Inspect the analyzer before blaming the ranking algorithm.

An inverted index avoids reading every document

Term	Documents containing the term
solar	D1, D3
battery	D2, D3
storage	D2

Query `solar AND battery`: intersect the lists to obtain D3.

Decision: Understand why exact-term retrieval is efficient and inspectable.

TF–IDF weights informative matches — 1 of 3

1. Count matching terms	Term frequency records how strongly a word appears in a passage. The query and passage use the same analyzed vocabulary.

Why this matters: Illustration: with the displayed unsmoothed IDF, a term in every document receives zero IDF.

TF–IDF weights informative matches — 2 of 3

1. Count matching terms	Term frequency records how strongly a word appears in a passage. The query and passage use the same analyzed vocabulary.
2. Give rare terms more discrimination	A word in every document helps little. A rare identifier can distinguish the requested item from many otherwise similar passages.

Why this matters: Illustration: with the displayed unsmoothed IDF, a term in every document receives zero IDF.

TF–IDF weights informative matches — 3 of 3

1. Count matching terms	Term frequency records how strongly a word appears in a passage. The query and passage use the same analyzed vocabulary.
2. Give rare terms more discrimination	A word in every document helps little. A rare identifier can distinguish the requested item from many otherwise similar passages.
3. Compare weighted representations	Cosine compares query and passage vectors with length normalization. The result is a ranking score, not a truth probability.

Why this matters: Illustration: with the displayed unsmoothed IDF, a term in every document receives zero IDF.

TF–IDF: read the score and its assumptions

$$w_{t,d} = \mathrm{tf}(t,d) \log \frac{N}{\mathrm{df}(t)}, \qquad s(q,d) = \frac{\mathbf{w}_q^\top \mathbf{w}_d}{\|\mathbf{w}_q\| \, \|\mathbf{w}_d\|}$$

t: term; d: document; q: query; N: collection size. $\mathrm{tf}(t,d)$: term count; $\mathrm{df}(t)$: documents containing that term.

$w_{t,d}$: term weight; $\mathbf{w}_q, \mathbf{w}_d$: weighted vectors. $\top$: transpose; $\|\cdot\|$: Euclidean length; s: cosine similarity.

The displayed IDF is unsmoothed and requires $\mathrm{df}(t) > 0$. The notebook uses smoothed IDF $\log((N+1)/(\mathrm{df}+1)) + 1$. Both rank lexical matches, not truth.

BM25 limits the benefit of repeated words — 1 of 3

1. Repetition should have diminishing value	Mentioning a query word one hundred times should not automatically beat a short paragraph that directly answers the question.

Why this matters: BM25 is a transparent lexical baseline; it does not resolve every synonym or qualification.

BM25 limits the benefit of repeated words — 2 of 3

1. Repetition should have diminishing value	Mentioning a query word one hundred times should not automatically beat a short paragraph that directly answers the question.
2. Account for document length	Long documents naturally contain more words. BM25 adjusts frequency relative to document length and the collection average.

Why this matters: BM25 is a transparent lexical baseline; it does not resolve every synonym or qualification.

BM25 limits the benefit of repeated words — 3 of 3

1. Repetition should have diminishing value	Mentioning a query word one hundred times should not automatically beat a short paragraph that directly answers the question.
2. Account for document length	Long documents naturally contain more words. BM25 adjusts frequency relative to document length and the collection average.
3. Tune two explicit choices	k1 controls frequency saturation; b controls length adjustment. Validate both on representative queries rather than treating a score as confidence.

Why this matters: BM25 is a transparent lexical baseline; it does not resolve every synonym or qualification.

BM25: read the score and its assumptions

$$s(q,d) = \sum_{t \in q} \mathrm{IDF}(t) \frac{f_{t,d}(k_1+1)}{f_{t,d} + k_1(1 - b + b|d|/\overline{L})}$$

q: query; d: document; t: query term. $\Sigma_{t \in q}$: add term contributions. $\mathrm{IDF}(t)$: chosen rarity weight.

$f_{t,d}$: term frequency; $|d|$: document length; $\overline{L}$: average collection length.

$k_1 > 0$: frequency saturation; $b \in [0,1]$: length adjustment. $s(q,d)$: ranking score.

Repetition has diminishing benefit. The notebook uses TF–IDF cosine, not BM25.

Question: can lexical search miss the right passage?

A useful passage expresses the same idea with different words. Which conclusion follows?

- A. Retrieval is impossible.
- B. Exact term matching may miss it; expansion or learned representations may help.
- C. Any dense retriever is guaranteed to recover it.

Decision: Choose a remedy that addresses the observed vocabulary mismatch.

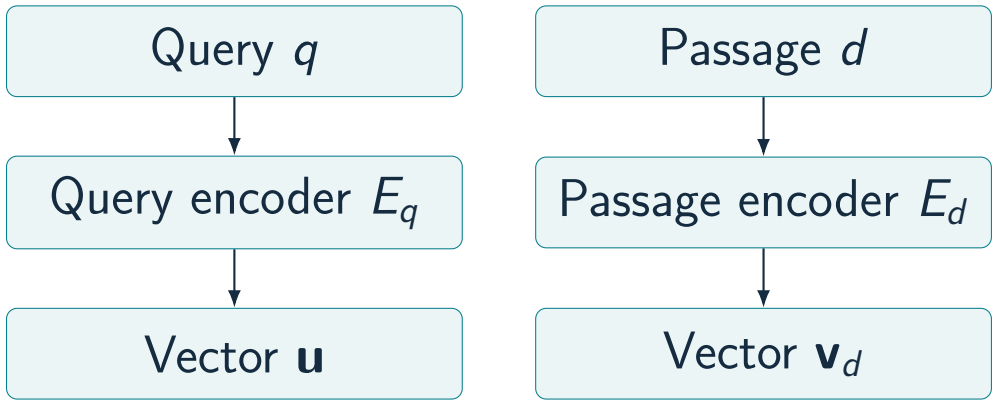
Answer: test a representation that bridges the wording

B. Expansion or learned representations may help.

- Keep exact-term retrieval as a baseline.
- Test paraphrases alongside entity, number and negation distinctions.
- Measure both recovered relevant passages and newly introduced errors.

Decision: A method is useful when it fixes a measured failure at acceptable cost.

A bi-encoder makes passage vectors reusable



$$\mathbf{u} = E_q(q), \quad \mathbf{v}_d = E_d(d)$$

$$s(q, d) = \mathbf{u}^\top \mathbf{v}_d$$

q: query text; d: passage text. E_q, E_d: learned encoders. $\mathbf{u}, \mathbf{v}_d$: vectors in the same space.

s: similarity; $\top$: transpose, making a dot product. Store passage vectors; encode each new query.

Optional notebook C07: actual local embeddings; default skips the model download.

Dense retrieval learns which passages belong together — 1 of 3

1. Supply a labelled comparison	Record: query=current meeting day; positive=the Friday update; negative=the superseded Tuesday notice. These are relevance labels.

Why this matters: The training set needs useful near-misses, not only unrelated passages.

Dense retrieval learns which passages belong together — 2 of 3

1. Supply a labelled comparison	Record: query=current meeting day; positive=the Friday update; negative=the superseded Tuesday notice. These are relevance labels.
2. Train relative similarity	The loss encourages the positive passage to score above the sampled negatives. The examples define which distinctions the representation learns.

Why this matters: The training set needs useful near-misses, not only unrelated passages.

Dense retrieval learns which passages belong together — 3 of 3

1. Supply a labelled comparison	Record: query=current meeting day; positive=the Friday update; negative=the superseded Tuesday notice. These are relevance labels.
2. Train relative similarity	The loss encourages the positive passage to score above the sampled negatives. The examples define which distinctions the representation learns.
3. Test the learned distinctions	False negatives, date changes and domain mismatch can cause errors. Nearby vectors do not by themselves prove relevance or truth.

Why this matters: The training set needs useful near-misses, not only unrelated passages.

Dense retrieval: read the loss and its assumptions

$$\mathcal{L} = -\log \frac{\exp(s(q, d^+)/\tau)}{\exp(s(q, d^+)/\tau) + \Sigma_j \exp(s(q, d_j^-)/\tau)}$$

q: query; d^+: labelled relevant passage; d_j^-: negative passage j. s: similarity score; $\tau > 0$: temperature.

exp: exponential; log: natural logarithm; Σ_j: sum over sampled negatives. $\mathcal{L}$: loss minimized during training.

The positive competes against this sampled set. Lower loss rewards its relative score; incorrect negatives teach the wrong distinction.

The optional notebook loads an existing encoder; it does not train this loss.

Negative examples determine the difficult distinctions

Negative type	What it teaches / risks
Random unrelated passage	Broad topic discrimination; may be too easy.
High-ranked but irrelevant passage	Fine distinctions; requires careful labels.
Relevant passage wrongly labelled negative	Incorrectly pushes useful evidence away.

Decision: Curate near-misses that resemble real retrieval failures.

Approximate search introduces a second recall question

Query vector $\longrightarrow$ vector index $\longrightarrow$ candidate IDs

- An exact scan compares the query with every vector.
- Approximate nearest-neighbour search trades speed and memory against exact top-result recovery.
- An exact nearest neighbour can still be irrelevant to the task.

Decision: Measure index recall separately from task relevance recall.

Hybrid retrieval combines different candidate signals

$$\mathrm{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{c + \mathrm{rank}_r(d)}$$

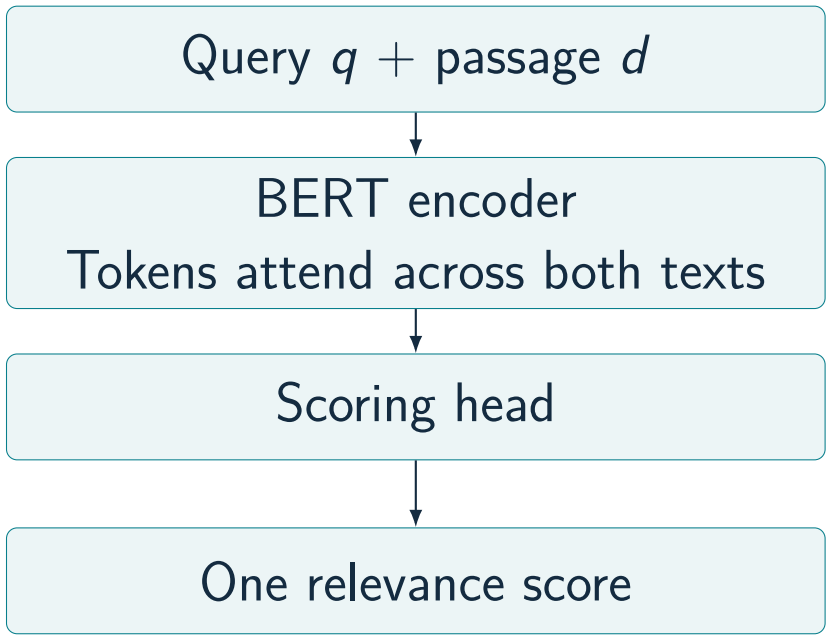
Reciprocal rank fusion: combine list positions instead of adding unlike raw scores.

d: candidate document; $\mathcal{R}$: set of ranked lists; r: one list. $\mathrm{rank}_r(d)$: its position, starting at 1.

$c > 0$: smoothing constant; Σ: add contributions from lists containing d. A missing document contributes zero.

Example with $c = 60$: ranks 1 and 3 give $1/61 + 1/63 \approx 0.0323$. This is a ranking score, not probability.

A cross-encoder reads the query and passage jointly



BERT: Bidirectional Encoder Representations from Transformers. It reads both left and right context within the supplied pair.

$$s_{\text{cross}}(q, d) = f_\theta([q; d])$$

$[q; d]$: query and passage joined in one input.
f_θ: learned scoring model; θ: its parameters.
s_{cross}: pair score.

Run once per candidate pair; use a shortlist.
The score is not automatically calibrated.

The shortlist creates a ceiling for reranking — 1 of 3

1. First-stage search creates candidates	If the required passage is not in that set, it is unavailable to a pure reranker.

Why this matters: Do not replace the generator to repair evidence that never reached it.

The shortlist creates a ceiling for reranking — 2 of 3

1. First-stage search creates candidates	If the required passage is not in that set, it is unavailable to a pure reranker.
2. Reranking orders what it receives	A stronger comparison model can promote a useful candidate. It cannot introduce a passage absent from its input.

Why this matters: Do not replace the generator to repair evidence that never reached it.

The shortlist creates a ceiling for reranking — 3 of 3

1. First-stage search creates candidates	If the required passage is not in that set, it is unavailable to a pure reranker.
2. Reranking orders what it receives	A stronger comparison model can promote a useful candidate. It cannot introduce a passage absent from its input.
3. Repair the earliest failed stage	Check collection and retrieval for a missing candidate; check reranking for a low-ranked candidate; check interpretation if context is already correct.

Why this matters: Do not replace the generator to repair evidence that never reached it.

Query rewriting helps only if intent is preserved

- Rewrite vague wording or expand useful synonyms.
- Split a compound information need into explicit subqueries.
- Retain entities, dates, negation and constraints.
- Record the original query, rewrite and returned evidence.

Decision: Evaluate rewrites as interventions, not hidden preprocessing.

Multiple retrieval steps require explicit stopping rules

Query → passages → identified gap → follow-up query

- A follow-up should address a named missing fact or relationship.
- Bound the number of queries, latency and spending.
- Stop with an unresolved gap when no eligible support is found.

Decision: Use iterative retrieval when later information needs depend on earlier results.

Context construction is a selection problem — 1 of 3

1. Search output is not yet model input	The application chooses which candidate passages to supply. The generator cannot read candidates omitted from its context.

Why this matters: A larger context window is capacity, not proof that every useful passage was selected or used.

Context construction is a selection problem — 2 of 3

1. Search output is not yet model input	The application chooses which candidate passages to supply. The generator cannot read candidates omitted from its context.
2. Spend the input budget deliberately	Keep controlling clauses, exceptions and source IDs. Duplicates and obsolete versions can crowd out essential information.

Why this matters: A larger context window is capacity, not proof that every useful passage was selected or used.

Context construction is a selection problem — 3 of 3

1. Search output is not yet model input	The application chooses which candidate passages to supply. The generator cannot read candidates omitted from its context.
2. Spend the input budget deliberately	Keep controlling clauses, exceptions and source IDs. Duplicates and obsolete versions can crowd out essential information.
3. Inspect the assembled input	Count actual tokens including instructions and wrappers. Save the selected text and tool results, not only the search list.

Why this matters: A larger context window is capacity, not proof that every useful passage was selected or used.

Historical RAG-Sequence: marginalizing candidate documents

Historical RAG-Sequence architecture: mix answer likelihoods over a retrieved document variable.

$$p(y \mid q) \approx \sum_{d \in \mathrm{TopK}(q)} p_\eta(d \mid q)\, p_\theta(y \mid q, d)$$

q: query; y: complete answer token sequence; d: candidate document.

$\mathrm{TopK}(q)$: selected K documents; Σ: add their contributions.

η: retriever parameters; θ: generator parameters. Each term multiplies a retrieval weight by an answer likelihood given that document.

Practical distinction: many RAG services concatenate passages into one prompt. They do not compute this mixture. Neither approach automatically gives a calibrated probability of truth.

Make the actual model input inspectable

Input component	Inspection question
Task and original question	What answer is being requested?
Selected source text and IDs	What evidence can the model actually see?
Instructions for uncertainty and citations	What response behavior is required?
Tool outputs, if any	Which computations were actually executed?

Decision: Store the assembled context, not only search results.

Notebook exercise: C08–C09. Read the three queries and the exact assembled prompt.

Retrieval can bring instructions as well as evidence

- Source text may contain commands addressed to a reader or model.
- Treat retrieved material as data within the intended task.
- Keep tool permissions and action checks outside source-controlled content.
- Test hostile or misleading passages as part of evaluation.

Decision: Do not let a source redefine the task or grant authority.

Support is different from citation presence

Check	What it establishes
Source exists	The cited record can be found.
Quoted text matches	The words appear in the source.
Claim is supported	The passage establishes the claim with its qualifications.
Source is reliable and applicable	A separate source-quality and scope judgement.

Decision: Evaluate citation correctness and claim support independently.

Evidence graphs organize declared dependencies

Source span → extracted fact → computation → claim

- Record which inputs each claim or calculation requires.
- Re-evaluate affected descendants when a source changes or disappears.
- Treat the graph as a record of declared dependencies; test whether those links are justified.

Decision: Use a graph to localize missing support and review impact.

Notebook exercise: C10–C13. Remove ANNEX, inspect the blocked path, then restore it.

Evaluate the stages with a fixed test collection

You need	Why
Representative queries and information needs	Define the target population of tasks.
Relevant passages and no-answer cases	Measure misses and unsupported completion.
Held-out queries and fixed corpus versions	Separate tuning from evaluation.
Recorded configurations and context	Reproduce and compare pipeline behavior.

Decision: Build a test collection before optimizing the retrieval stack.

Precision and recall measure different costs — 1 of 3

1. Count the relevant returned items	Suppose two of three returned passages are relevant, and four relevant passages exist in the labelled collection.

Why this matters: These are retrieval-list measures. Evaluate the generated answer separately.

Precision and recall measure different costs — 2 of 3

1. Count the relevant returned items	Suppose two of three returned passages are relevant, and four relevant passages exist in the labelled collection.
2. Precision asks how clean the list is	Two useful results out of three returned gives precision 2/3. Irrelevant results can distract the generator.

Why this matters: These are retrieval-list measures. Evaluate the generated answer separately.

Precision and recall measure different costs — 3 of 3

1. Count the relevant returned items	Suppose two of three returned passages are relevant, and four relevant passages exist in the labelled collection.
2. Precision asks how clean the list is	Two useful results out of three returned gives precision 2/3. Irrelevant results can distract the generator.
3. Recall asks how much was recovered	Two found out of four labelled relevant items gives recall 1/2. A missing exception may matter even with high precision.

Why this matters: These are retrieval-list measures. Evaluate the generated answer separately.

Precision and recall: read the denominators

$$\text{Precision@k} = \frac{|R_k \cap G|}{k}, \qquad \text{Recall@k} = \frac{|R_k \cap G|}{|G|}$$

k: requested cutoff; R_k: returned distinct top-k items; G: labelled relevant items.

$\cap$: intersection; $|\cdot|$: number of items. The numerator counts returned relevant items.

Example: 2 relevant of 3 returned, with 4 relevant in the collection: precision $2/3$, recall $2/4$.

Specify empty-result handling. The notebook divides by requested k even if fewer than k results return.

Notebook exercise: C14. Inspect Retrieved, Recall and Missed; change only k.

MRR: how soon do we find the first useful result?

$$\mathrm{MRR} = \frac{1}{Q}\sum_{i=1}^{Q} \mathrm{RR}_i, \qquad \mathrm{RR}_i = \begin{cases} 1/r_i & \text{if a relevant result is found,} \\ 0 & \text{otherwise.} \end{cases}$$

Mean reciprocal rank (MRR): average reward for finding the first useful result early.

Q: number of evaluated queries; i: query index; r_i: rank of its first relevant result (starting at 1). RR_i: that query's reciprocal rank.

Ranks 1, 2 and no hit give $\mathrm{MRR} = (1 + 1/2 + 0)/3 = 0.50$.

Limit: later relevant passages do not change MRR once the first hit is fixed.

nDCG: reward several useful results in the right order

$$\mathrm{DCG@k} = \sum_{j=1}^{k} \frac{2^{g_j} - 1}{\log_2(j+1)}, \qquad \mathrm{nDCG@k} = \frac{\mathrm{DCG@k}}{\mathrm{IDCG@k}}$$

k: cutoff; j: rank; g_j: relevance grade at that rank. $2^{g_j} - 1$: gain; $\log_2(j+1)$: position discount.

DCG: discounted cumulative gain. **IDCG:** DCG for the ideal grade ordering. **nDCG:** normalized DCG, from 0 to 1 when ideal gain is positive.

Grades [0,2,1] give $0 + 3/\log_2 3 + 1/2 \approx 2.393$. Ideal [2,1,0] gives $3 + 1/\log_2 3 \approx 3.631$. Thus nDCG ≈ 0.659.

Declare grades, cutoff and handling of zero ideal gain. This measures ordering against labels, not answer correctness.

One small example makes the trade-off visible

Top three relevance labels	Precision@3	Recall@3
Relevant, irrelevant, relevant	2/3	2/4
Relevant, relevant, relevant	3/3	3/4

Assume four labelled relevant passages in the collection.

Neither result set establishes complete coverage.

Decision: Read the numerator and denominator before interpreting a metric.

Evaluate answers separately from the retrieval list

Dimension	Question
Correctness	Does the answer agree with the adjudicated target?
Groundedness	Are its claims supported by supplied evidence?
Citation quality	Do references support the linked claims?
Abstention	Does it acknowledge insufficient evidence?
Efficiency	What latency and cost were needed?

Decision: A single answer score cannot explain why a system failed.

Controlled comparisons reveal which stage matters

Intervention	Question it isolates
Supply adjudicated supporting passages	Can the reader answer with good evidence?
Hold the reader fixed; change retriever	Does candidate selection improve?
Hold passages fixed; change order	Is context use position-sensitive?
Remove a required passage	Does the system detect the evidence gap?

Decision: Change one component or evidence condition at a time.

Similarity and confidence require different validation — 1 of 3

1. A score orders candidates	Its value depends on the representation, model and candidate set. A cosine score of 0.9 is not 90 percent confidence in an answer.

Why this matters: Define the event before interpreting any score as a probability.

Similarity and confidence require different validation — 2 of 3

1. A score orders candidates	Its value depends on the representation, model and candidate set. A cosine score of 0.9 is not 90 percent confidence in an answer.
2. Normalization does not establish calibration	Numbers summing to one can still be poor estimates of a defined real-world event.

Why this matters: Define the event before interpreting any score as a probability.

Similarity and confidence require different validation — 3 of 3

1. A score orders candidates	Its value depends on the representation, model and candidate set. A cosine score of 0.9 is not 90 percent confidence in an answer.
2. Normalization does not establish calibration	Numbers summing to one can still be poor estimates of a defined real-world event.
3. Validate any decision threshold	Use reviewed cases to measure coverage and admitted errors. Recheck after source, model or task changes.

Why this matters: Define the event before interpreting any score as a probability.

Question: where should the repair begin?

A reranker never receives the required passage. Which change directly addresses this failure?

- A. Improve the collection or candidate retrieval.
- B. Ask the generator to sound more certain.
- C. Raise the reranker score threshold.

Decision: Locate the first missing dependency.

Answer: repair the candidate path first

A. Check collection coverage, then candidate retrieval.

- If the source is absent, acquire or index it.
- If it is indexed, inspect query representation and retrieval settings.
- Only then assess whether reranking and reading preserve the useful evidence.

Decision: Use the smallest intervention that addresses the observed failure.

Select the architecture from observed requirements

Observed requirement	Candidate design choice
Exact names, phrases and identifiers	Strong lexical baseline and appropriate analyzer.
Frequent paraphrases	Evaluate dense or hybrid retrieval.
Subtle query-passage compatibility	Evaluate a cross-encoder reranker.
Dependent information needs	Bounded iterative retrieval.
Strict historical or access constraints	Enforced filters and versioned source capture.

Decision: Choose components to fix measured failure modes under a cost budget.

The roadmap now becomes a diagnostic checklist

- Can the required evidence be found in the indexed collection?
- Does the query representation retrieve it?
- Do filters, reranking and context selection retain it?
- Does the reader use it with the right qualifications?
- Do held-out tests support the performance claim?

Decision: Explain a failure by stage, then design the next test.

Lecture sources: retrieval foundations

Material	Role in this lecture
Manning, Raghavan and Schuetze: Introduction to Information Retrieval	Chapters 1–2: indexing; 6: scoring; 8: evaluation; 9: expansion; 11: BM25.
Stanford CS224N 2025, Lecture 13	QA taxonomy, retriever–reader framework and retrieval models.
Karpukhin et al., EMNLP 2020	Dense passage encoders, training and negatives.

Decision: Use these references to deepen the mechanism you need to change.

Lecture sources: RAG and context use

Material	Role in this lecture
Sentence Transformers: Retrieve and Re-Rank	Bi-encoder / cross-encoder architecture and diagrams.
Lewis et al., NeurIPS 2020	Retrieval-conditioned generation and RAG-Sequence.
Liu et al., TACL 2024	Controlled context-position experiment.
Microsoft Azure AI Search documentation	Hybrid retrieval, rank fusion and filters.

Decision: Use primary results with their model, task and implementation limits.